Специальность: 5-04-0612-02 «Разработка и сопровождение программного обеспечения информационных систем»	Учебная дисциплина: «Основы кроссплатформенного программирования»

Лабораторная работа № 3
Инструкционно-технологическая карта

Тема: Разработка, отладка и испытание программ с использованием строк

Цель: Научиться работать с классамиString, StringBuilder, StringBuffer, создавать строки, искать символы в строке, заменять символы в строках, менять регистр, определять вхождение символа.

Время выполнения: 2 часа

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Изучить теоретические сведения;
2. Ответить на контрольные вопросы;
3. Откомпилировать примеры программ из раздела «Теоретические сведения»;
4. Выполнить ИДЗ.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ДЛЯ ВЫПОЛНЕНИЯ РАБОТЫ

Строка – это упорядоченная последовательность символов. В Java строка является основным носителем текстовой информации. Для работы со строками здесь используются следующие классы: String, StringBuilder, StringBuffer.

Создание строк

Строка в Java является объектом, поэтому ее можно создать, как и любой другой объект, при помощи оператора new.

String str1 = new String ("Строка созданная при помощи оператора new");
Также строку можно создать при помощи литерала (фразы заключенной в кавычки) следующим образом.

String str2 = "Эта строка создана при помощи литерала.";

Обе строки, независимо от способа создания являются объектами – экземплярами класса String.

Важный момент: создание объектов при помощи литерала возможно только в классе String. Объекты любого другого класса при помощи литерала создать нельзя.

Можно также создать массив строк. Например, так:

String[] auto = {"Маша", "Петя", "Вася", "Коля"};

Конкатенация или слияние строк в Java

Для того, чтобы объединить несколько разных строк в одну, в Java можно использовать перегруженные (специально для объектов String) операторы «+» и «=+».

Еще один важный момент: операторы «+» и «=+», перегруженные для String, являются единственными перегруженными операторами в Java. Программист здесь не имеет возможности самостоятельно перегружать какие либо операторы (как, например, в C++ и некоторых других языках).

Пример 1:

```
String str1 = "Мама ";
String str2 = "мыла ";
String str3 = "раму";

String result = str1 + str2 + str3;
System.out.print(result);
```

На консоль будет выведено «Мама мыла раму»

Пример 2:

```
String[] animals = {"Хаски","Морж"};// массив строк 1
String[] food = {"колбаски","корж"}; // массив строк 2

//составляем строки из элементов массивов и связующего слова
String result1 = animals[0] + " ест " + food[0];
String result2 = animals[1] + " ест " + food[1];

//выводим на консоль
System.out.println(result1);
System.out.println(result2);
```

Пример 3:

```
String[] auto = {"Волга", "Чайка", "Жигули"}; //задан массив строк
String result = "В гараже стоят машины: "; //задана строка

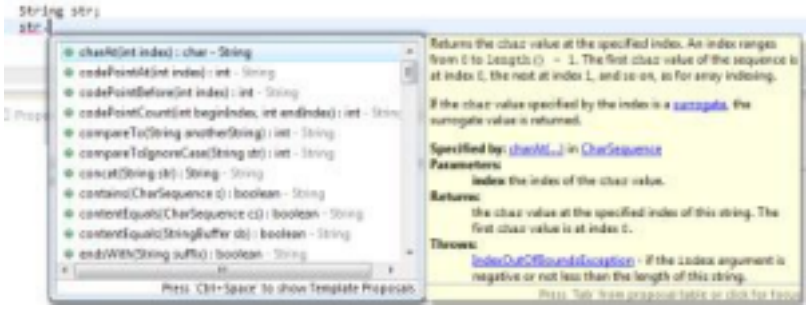
//прибавляем к строке элементымассива
for(int i = 0; i < auto.length; i++){
//если элемент не последний, разделяем запятой
if(i != auto.length-1 )
result += auto[i] + ", ";
//если последний, ставим после него точку
else
result += auto[i] + ".";
 }
//выводим результат
System.out.print(result);
```

Наиболее употребительные методы класса String

При использовании IDE можно легко увидеть, какие методы есть у класса и получить подсказку по их использованию. На примере IDE Eclipse: для того,

чтобы открыть список методов и быстро выбрать подходящий, нужно после

имени переменной поставить точку и нажать комбинацию клавиш CTRL + Space (пробел). После этого появится окно, как на рисунке, где будут перечислены все доступные методы класса.



При выборе метода из этого списка, справа (или слева) появится желтое окно с подсказкой по его использованию. При помощи нажатия Enter или двойного клика мыши метод можно вставить в ваш код, не прибегая к ручному набору.

Также после имени переменной и точки можно начать набирать вручную имя метода и после введения нескольких первых букв нажать CTRL + Space (пробел). При этом, если метод, начинающийся на эти буквы один, то он автоматически подставится в код, если методов несколько, то откроется окно, как на рисунке, где будут перечислены только те методы, которые начинаются с этих введенных вами букв.

Далее рассмотрим методы, которые чаще всего используются при работе со строками. Некоторые задачи можно решить и без применения этих методов, но их знание значительно облегчает процесс программирования. В дальнейшем описании, первое слово, которое стоит перед названием метода – тип значения, которое возникнет в результате работы метода (значение, которое метод возвращает).

Конкатенация

String concat(String str) – производит ту же конкатенацию, что была описана выше, но использование этого метода из класса String положительно влияет на производительность и скорость программы. На небольших примерах это незаметно и не существенно, но в более серьезных приложениях стоит использовать этот метод. Результатом работы метода будет строка. Параметр, который нужно передавать в метод для конкатенации – тоже строка, о чем нам говорит значение в скобках (String str).

Перепишем пример 2, при помощи concat():

```
String[] animals = {"Хаски","Морж"};// массив строк 1
String[] food = {"колбаски","корж"}; // массив строк 2

//составляем строки из элементов массивов и связующего слова
String result1 = animals[0].concat(" ест ").concat(food[0]);
String result2 = animals[1].concat(" ест ").concat(food[1]);
```

```
//выводим на консоль
```


```
System.out.println(result1);
System.out.println(result2);
```

Определение количества символов в строке

Для того чтобы определить количество символов в строке, используется метод length.

int length() – возвращает длину строки. Длина равна количеству символов Unicode в строке.

```
String str = "Строка из букв, цифр 492 и специальных символов %*;№?";
int length = str.length();
System.out.println("Длина строки = " + length);
```

Извлечение символов из строки

Если нам требуется узнать, какой символ находиться в строке на конкретной позиции, можем использовать метод charAt.

char charAt(int index) – возвращает символ, находящийся по указанному индексу в строке. Результатом работы метода будет символ типа char. Параметр, который передается в метод – целое число. Первый символ в строке, подобно массивам, имеет индекс 0.

Пример 5: определить последний символ в строке.

```
String str = "Последний символ в этой строке - о";
int last = str.length()-1;//длина строки - 1, так как отсчет начинается с 0
char ch = str.charAt(last);
System.out.println(ch);
```

Если мы хотим работать со строкой, как с массивом символов, можем конвертировать строку в массив при помощи метода toCharArray. char[] toCharArray() – преобразует строку в новый массив символов. Пример 6: поменять в строке символы пробела на точки при помощи преобразования в массив символов (для этой задачи есть более простое решение, нежели преобразование в массив, но об этом чуть позже). Примечание: в данном случае мы не сможем использовать метод charAt. При помощи этого метода мы бы смогли только найти пробелы в строке, но не поменять их.

```
String str = "1 000 000 000";
//преобразовываем строку в массив
char[] chArray = str.toCharArray();
//перебираем все элементы массива
for(int i = 0; i<chArray.length; i++){
//находим пробел
if(chArray[i] == ' '){
//заменяем на точку
chArray[i] = '.';
} }
//выводим результат
System.out.println(chArray);
```

Извлечение подстроки из строки

String substring(int beginIndex, int endIndex) или substring(int beginIndex) – возвращает новую строку, которая является подстрокой используемой строки. В параметрах метода нужно указать индекс строки, с которого начинается подстрока и индекс, которым заканчивается. Также возможно указывать только начальный индекс. В этом случае будет возвращена подстрока от начального индекса и до конца строки.

```
String s = "www.mysite.com";
String name = s.substring(4,s.length()-4);
System.out.println(name); // на консоль выведет "mysite"

String domain = s.substring(4);
System.out.println(domain); // на консоль выведет "mysite.com"
```

Разбиение строк

Для разбиения строк на части используется метод String[] split(String regex), который разбивает строку на основании заданного регулярного выражения.

```
String isbn = "978-3-16-148410-0";
String[] isbnParts = isbn.split("-");

System.out.println("префикс EAN.UCC: " + isbnParts[0]);
System.out.println("номер регистрационнойгруппы: " + isbnParts[1]);
System.out.println("номер регистранта: " + isbnParts[2]);
System.out.println("номер издания: " + isbnParts[3]);
System.out.println("контрольная цифра: " + isbnParts[4]);
```

Поиск в строке

boolean contains(CharSequence s) – проверяет, содержит ли строка заданную последовательность символов и возвращает true или false.

```
String s = "www.mysite.com";
boolean isContain1 = s.contains("mysite");
System.out.println(isContain1);// нашел - выведет true
boolean isContain2 = s.contains("mysite.ru");
System.out.println(isContain2);// не нашел - выведет false
```

boolean endsWith(String suffix) – проверяет завершается ли строка определенными символами и возвращает true или false.

```
String s = "www.mysite.com";
//проверяем заканчивается ли строка суффиксом "com"
boolean isComEnding = s.endsWith("com");
System.out.println(isComEnding);//выведет true
//проверяем заканчивается ли строка суффиксом "ru"
boolean isRuEnding = s.contains("ru");
System.out.println(isRuEnding);//выведет false
```

boolean startsWith(String prefix) или startsWith(String prefix, int toffset) – проверяет, начинается ли строка с определенных символов. Во втором случае можно указать позицию с которой необходимо начать поиск префикса. String s =

```
"www.mysite.com";

    //Проверяем, начинается ли адрес с www
```

```
    boolean isWWW = s.startsWith("www");

    if(isWWW){
    /* Если да, проверяем начинается ли имя сайта
    с "my". Поскольку адрес начинается с www
    проверку начинаем с 4 символа*/
    boolean isNameStarts = s.startsWith("my", 4);
    }else{
    /* Если нет, проверяем начинается ли имя сайта
    с "my". Поскольку адрес не начинается с www
    проверку производим с начала строки*/
    boolean isNameStarts = s.startsWith("my");
    }
```

int indexOf(int ch), indexOf(int ch, int fromIndex), indexOf(String str), indexOf(String str, int fromIndex) – метод indexOf применяется для поиска первого вхождения указанного символа в строке или первого вхождения указанной подстроки. Поиск также можно произвести с указанием позиции в строке от которой нужно начинать искать. Для поиска нужно указать соответствующие параметры. Метод возвращает число соответствующее индексу первого вхождения символа или подстроки. В случае отсутствия указанного символа или подстроки в строке, будет возвращена -1.

```
    String data = "name:Igor\nsurname:Kolashnikov\nage:14\ntime:14:55";
    //разбиваем строку на несколько подстрок на основании
    // встречаемого символа новой строки
    String[]lines=data.split("\n");

    //проходим каждую подстроку
    for (String line : lines){
    //находим индекс первого вхождения символа ":" в подстроке
    int pos = line.indexOf(":");
    //вычленяем имя атрибута из подстроки
    String attributeName= line.substring(0,pos);
    //вычленяем значение атрибута
    String value = line.substring(pos+1,line.length());
    //вывод на экран вычлененных значений в нужном нам формате.
    System.out.println(attributeName + " - " + value);
    }
```

int lastIndexOf(int ch), lastIndexOf(int ch, int fromIndex), lastIndexOf(String str), lastIndexOf(String str, int fromIndex) – аналогично предыдущему случаю, только ищется последнее вхождение символа или подстроки в строке. Модификация строк

Модификация строк не является модификацией как таковой. Дело в том, что объекты класса String после создания уже нельзя изменять. Но можно создать

копию строки с изменениями. Именно это и делают следующие методы. toLowerCase() – преобразовать строку в нижний регистр; toUpperCase() – преобразовать строку в верхний регистр; trim() – отсечь на концах строки пустые символы;

Пример 13

```
String str = " Я помню ЧУДНОЕ мгновенье ";

//убрали символы пробела в начале и конце строки
str = str.trim();

//я помню чудное мгновенье
System.out.println(str.toLowerCase());

//Я ПОМНЮ ЧУДНОЕ МГНОВЕНЬЕ
System.out.println(str.toUpperCase());
```

String replace(char oldChar, char newChar), replace(CharSequence target, CharSequence replacement) – замена в строке одного символа или подстроки на другой символ или подстроку.

Вспомним пример 6, где нужно было поменять в строке символы пробела на точки и перепишем его с использованием replace:

```
String str = "1 000 000 000";
String newStr = str.replace(" ", ".");
System.out.println(newStr);
```

Сравнение строк

boolean equals(Object anObject) – проверяет идентичность строк. Возвращает true только в том случае, если в строках представлена одинаковая последовательность символов одной величены.

Пример 14

```
String str = "Я помню ЧУДНОЕ мгновенье";
String str2 = "я помню чудное мгновенье";
//строки не идентичны
System.out.println(str.equals(str2)); //false
//строки идентичны после перевода первой строки
//в нижний регистр
System.out.println(str.toLowerCase().equals(str2)); // true
```

int compareTo(String anotherString) – так же проверяет идентичность строк, однако, в отличии от метода equals возвращает:

- нулевое значение, если строки равны,
- целое отрицательное число, если первая строка предшествует второй
- целое положительное число, если первая строка следует за второй

Данный метод предназначен для упорядочивания строк. Он позволяет сравнить строки между собой и определить предшествующую строку. Для того, чтобы реализовать такое сравнение метод сравнивает числовые значения

букв.

Для сравнения строк без учета регистра символов используется функция int compareToIgnoreCase(String str)

System.out.println("маша".compareToIgnoreCase("Миша")); //-8 Как мы видим, при сравнивании «маша» с «Миша» мы снова получаем отрицательное значение, то есть «маша» предшествует имени «Миша». Пример 16. Использование printf для форматирования в Java Метод printf() принадлежит классу PrintStream, который отвечает за вывод информации. Уже знакомые нам методы print() и println() также являются методами класса PrintStream.

Метод printf определен следующим образом:

printf(String format, Object... args)

Первый аргумент format это строка, определяющая шаблон, согласно которому будет происходить форматирование. Для ее записи существуют определенные правила.

В предыдущем примере формат был «%4d», где d означает вывод десятичного целого числа, а 4 – означает то, что если количество знаков в числе меньше, чем 4, то оно будет спереди дополнено пробелами на недостающее (до 4-х) количество знаков (тем самым подвинуто вправо).

Для наглядности приведем еще один пример, который выводит столбиком несколько чисел.

System.out.printf("%6d%n%6d%n%6d%n%6d%n%6d%n%6d",666666, 55555, 4444, 333, 22, 1);

На консоль будет выведено:

```
666666
 55555
 4444
 333
 22
 1
```

Регулярные выражения

Регулярные выражения или шаблоны используются для поиска подстроки или строки, соответствующей шаблону в строке, тексте или другом объекте, представляющем последовательность символов.

Класс java.util.regex.Pattern применяется для создания этого объекташаблона. Для определения шаблона применяются специальные синтаксические конструкции. О каждом соответствии можно получить информацию с помощью класса java.util.regex.Matcher.

Далее приведены некоторые логические конструкции для задания шаблона.

Эти же конструкции применяются не только в Java, но и в других распространенных языках.

При создании регулярного выражения могут использоваться логические

операции:

ab	после **a** следует **b**
a|b	**a** или **b**

Если необходимо, чтобы в строке, проверяемой на соответствие, в какой-либо позиции находился один из символов некоторого символьного набора, то такой набор (класс символов) можно объявить, используя одну из следующих конструкций:

[abc]	**a** или **b** или **c**
[^abc]	символ, исключая **a**, **b** и **c**
[a-z]	символ между **a** и **z**

Кроме стандартных классов символов существуют предопределенные классы символов:

. любой символ
\d или \p{Digit} [0-9]
\D [^0-9]
\s или \p{Space} [\t\n\x0B\f\r]
\S [^\s]
\w [0-9_A-Za-z]
\W [^\w]
\p{Lower} [a-z]
\p{Upper} [A-Z]
\p{Alpha} [a-zA-Z]
\p{Alnum} [\p{Alpha}\p{Digit}]
\p{Punct} !"#$%&'()*+,-./:;<=>?@[\]^_`{|}~ \p{Blank} пробел
или табуляция
^ или \A начало строки
$ или \Z конец строки

Круглые скобки, кроме их логического назначения, также используются для определения групп.

Для определения регулярных выражений недостаточно одних классов символов, т.к. в шаблоне часто нужно указать количество повторений. Для этого существуют квантификаторы.

a?	**a** один раз или ни разу
a*	**a** ноль или более раз
a+	**a** один или более раз
a{n}	**a** n раз
a{n,}	**a** n или более раз
a{n,m}	**a** от n до m

Класс Pattern используется для простой обработки строк и объекта хранителя регулярного выражения. Объект класса Pattern, в свою очередь, используется для более сложной обработки символьной информации классом Matcher, рассмотренном несколько позже.

Некоторые методы класса Pattern:

static Pattern compile(String regex) – возвращает Pattern, который соответствует regex;

static boolean matches(String regex, CharSequence input) – проверяет на соответствие строки input шаблону regex;

String[] split(CharSequence input) – разбивает на массив строку input, учитывая, что разделителем является шаблон;

Stream<String> splitAsStream(CharSequence input) – разбивает в stream строку input, учитывая, что разделителем является шаблон; Predicate<String> asPredicate() – возвращает предикат на основе регулярного выражения;

Matcher matcher(CharSequence input) – возвращает Matcher, с помощью которого можно находить соответствия в строке input.

Методы класса Pattern позволяют проверять на соответствие шаблону целую строку и разбивать строку на части, используя шаблон как разделитель. Для определения подстрок, соответствующих шаблону, необходимо использовать класс Matcher.

Начальное состояние объекта типа Matcher не определено. Должен быть вызван первоначально один из методов поиска. Попытка вызвать какой-либо метод класса для извлечения информации о найденном соответствии приведет к возникновению ошибки IllegalStateException. Чтобы начать работу с объектом Matcher, необходим вызов одного из следующих методов: boolean lookingAt() – поиск последовательности символов, начинающейся с начала строки и соответствующей шаблону; boolean find() или boolean find(int start) – определение последовательности символов, соответствующих шаблону, в любом месте строки. Параметр start указывает на начальную позицию поиска. После вызова одного из этих методов можно воспользоваться и методами о состоянии объекта.

Сбросить состояние экземпляра Matcher в исходное, для этого применяется метод reset() или reset(CharSequence input), который также заменяет на новую последовательность символов для обработки.

Метод boolean matches() проверяет, соответствует ли вся информация шаблону.

Для замены всех подпоследовательностей символов, удовлетворяющих шаблону, на заданную строку можно применить метод replaceAll(String replacement).

Следующие примеры показывают использование возможностей классов Pattern и Matcher для поиска, разбора и разбивки строк.

Проверка на соответствие строки шаблону:

```
Pattern pattern = Pattern.compile("x+y");
Matcher matcher = pattern.matcher("xxxy");
boolean res = matcher.matches();
System.out.println(res); // printing true
```

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class RegexTasks {
 public static void main(String[] args) {
 String text = "aeeea aeea aea axa axxa axxxa";
 Pattern pattern = Pattern.compile("a[ex]+a"); // Меняем под задачу
Matcher matcher = pattern.matcher(text);

 while (matcher.find()) {
 System.out.println(matcher.group());
 }
 }
}
```

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое строка?
2. Как создается строка?
3. Как выглядит массив строк?
4. Что такое конкатенация?
5. Как происходит слияние строк?
6. Какие классы работают с регулярными выражениями?

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Выполнить задания по вариантам (для всех заданий номер варианта определяется номером в списке журнала группы).

Задание 1.

1. Вывести на экран длину введенной строки.
2. Вывести первый символ строки.
3. Вывести последний символ строки.
4. Вывести строку в верхнем регистре.
5. Вывести строку в нижнем регистре.
6. Проверить, является ли строка пустой (isEmpty()).
7. Проверить, состоит ли строка только из пробелов (isBlank()). 8. Сравнить две строки на полное совпадение (equals()).
9. Сравнить две строки без учета регистра (equalsIgnoreCase()). 10. Найти индекс первого вхождения символа 'a' в строке (indexOf). 11. Найти индекс последнего вхождения символа 'o' в строке (lastIndexOf).
12. Проверить, начинается ли строка с подстроки "Привет" (startsWith).
13. Проверить, заканчивается ли строка на ".txt" (endsWith). 14. Заменить все буквы 'e' на 'и' в строке (replace).

15. Заменить все пробелы на символ подчеркивания (replace).

16. Удалить все пробелы в начале и конце строки (trim).

17. Склеить две строки в одну через пробел (конкатенация +).

18. Извлечь подстроку с 3-го по 7-й символ (substring).

19. Извлечь подстроку с 0-го по 4-й символ.

20. Извлечь подстроку с 5-го символа до конца.

21. Получить символ по индексу 4 (charAt).

22. Преобразовать строку в массив символов (toCharArray()) и вывести их через пробел.

23. Проверить, содержит ли строка подстроку "Java" (contains).

24. Преобразовать число (int) в строку (String.valueOf()).

25. Преобразовать строку в число (Integer.parseInt()) и вывести его удвоенное значение.

26. Вывести на экран все символы строки в столбик (через цикл for).

27. Подсчитать количество символов 'o' в строке (цикл).

28. Подсчитать количество заглавных букв в строке.

29. Проверить, являются ли две строки "зеркальными" (например, "abc" и "cba" – использовать reverse() или цикл).

30. Вывести строку столько раз, сколько в ней символов (например, "Привет" -> вывести "Привет" 6 раз).

Задание 2.

1. Разбить предложение на слова и вывести каждое слово с новой строки.

2. Подсчитать количество слов в предложении.

3. Найти самое короткое слово в предложении.

4. Найти самое длинное слово в предложении.

5. Перевернуть каждое слово в предложении (например, "Hello World" -> "olleH dlroW").

6. Проверить, является ли строка палиндромом (с удалением пробелов и приведением к нижнему регистру).

7. Удалить все гласные буквы из строки (использовать replaceAll с регулярным выражением).

8. Подсчитать количество гласных и согласных букв в строке. 9. Извлечь из email (user@domain.com) имя пользователя (часть до @). 10. Извлечь из email доменную часть (часть после @).

11. Проверить, является ли строка корректным email (содержит @ и точку после @).

12. Проверить, является ли строка корректным номером телефона РБ (только цифры и + в начале).

13. Вывести инициалы из ФИО (например, "Иванов Петр Сидорович" -> "Иванов П. С.").

14. Сжать строку: заменить все повторяющиеся символы на один (например, "aaabbbcc" -> "abc").

15. Проверить, содержит ли строка только буквы и цифры

16. Найти все позиции (индексы) вхождения символа 'a' в строке.

17. Удалить все слова короче 3 символов из предложения.

18. Разбить предложение на слова и вывести каждое слово с новой строки.

19. Подсчитать количество слов в предложении.

20. Найти самое короткое слово в предложении.

21. Найти самое длинное слово в предложении.

22. Перевернуть каждое слово в предложении (например, "Hello World" -> "olleH dlroW").

23. Проверить, является ли строка палиндромом (с удалением пробелов и приведением к нижнему регистру).

24. Удалить все гласные буквы из строки (использовать replaceAll с регулярным выражением).

25. Подсчитать количество гласных и согласных букв в строке. 26.

Извлечь из email (user@domain.com) имя пользователя (часть до @). 27.

Извлечь из email доменную часть (часть после @).

28. Проверить, является ли строка корректным email (содержит @ и точку после @).

29. Проверить, является ли строка корректным номером телефона РБ (только цифры и + в начале).

30. Вывести инициалы из ФИО (например, "Смирнова Елена Игоревна" -> "Смирнова Е. И.").

Задание 3.

Создать класс Employee, у которого есть переменные класса - fullname, salary. Создать массив, содержащий несколько объектов этого типа. Создать класс Report, который будет содержать статический метод generateReport, в котором выводится информация о зарплате всех сотрудников. Используйте форматировании строк. Пусть salary будет выровнено по правому краю, десятичное значение имеет 2 знака после запятой. Выделить под фамилию 15 символов.

Задание 4.

1. Дана строка 'aeeea aeea aea aha axxa axxxa'. Найти и вывести строки следующего вида: по краям стоят буквы 'a', а между ними - или буква 'e' любое количество раз или по краям стоят буквы 'a', а между ними - буква 'x' любое количество раз.

2. Дана строка 'aeeea aeea aea aha axxa axxxa'. Найти и вывести строки следующего вида: по краям стоят буквы 'a', а между ними - или буква 'e' два раза или буква 'x' любое количество раз.

3. Дана строка 'aAXa aeffa aGha aza ax23a a3sSa'. Найти и вывести строки следующего вида: по краям стоят буквы 'a', а между ними - маленькие латинские буквы и цифры, не затронув остальных.

4. Дана строка “Versions: Java 5, Java 6, Java 7, Java 8,Java 12.” Найти все подстроки "Java X" и распечатать их.

5. Дана строка 'aAXa aeffa aGha aza ax23a a3sSa'. Найти и вывести строки следующего вида: по краям стоят буквы 'a', а между ними - маленькие и большие латинские буквы, не затронув остальных.

6. Предложение состоит из нескольких слов, разделенных пробелами. Например: "One two three раз два три one1 two2 123 ". Найти количество слов, содержащих только символы латинского алфавита.

7. Дана строка 'ааа ббб ёёё ззз ййй ААА БББ ЁЁЁ ЗЗЗ ЙЙЙ'. Найти и вывести все слова по шаблону: любая кириллическая буква любое количество раз.

8. Дана строка 'aAXa aeffa aGha aza ax23a a3sSa'. Найти и вывести строки следующего вида: по краям стоят буквы 'a', а между ними - маленькие латинские буквы, не затронув остальных.

9. Дана строка 'abba abea abca abza abxa'. Найти и вывести все строки, которые начинаются на 'ab' и заканчиваются на 'a', а между ними ровно одна любая буква (кроме 'b').

10. Дана строка 'xzzx xzxx xxxx xabx x12x xAax'. Найти и вывести строки, где по краям стоит 'x', а между ними – ровно две любые цифры. 11. Дана строка 'a1a a22a a333a a4444a a55555a'. Найти и вывести строки, где по краям стоит 'a', а между ними – цифры от 2 до 5 в любом количестве (от 1 до бесконечности).

12. Дана строка 'aba abba abbba abbbba'. Найти и вывести строки, где по краям стоит 'a', а между ними – буква 'b' ровно 3 раза.

13. Дана строка 'aAa aBa aCa aDa aEa'. Найти и вывести строки, где по краям стоит 'a', а между ними – **одна** заглавная латинская буква от A до D. 14. Дана строка 'a1a a22a a333a a4444a a55555a'. Найти и вывести строки, где по краям стоит 'a', а между ними – **только** четные цифры (0, 2, 4, 6, 8) в любом количестве.

15. Дана строка 'a!a a@a a#a a$a a%a'. Найти и вывести строки, где по краям стоит 'a', а между ними – **один** специальный символ (не буква и не цифра).

16. Дана строка 'apple ap-ple ap_ple ap.ple ap1ple'. Найти и вывести все слова, которые начинаются на 'ap' и заканчиваются на 'le', а между ними – ровно один любой символ (кроме пробела).

17. Дана строка 'a1a a2a a3a a4a a5a a6a a7a a8a a9a'. Найти и вывести строки, где по краям стоит 'a', а между ними – **одна** цифра от 1 до 5 включительно.

18. Дана строка 'aXa aXXa aXXXa aXXXXa aXXXXXa'. Найти и вывести строки, где по краям стоит 'a', а между ними – буква 'X' от 2 до 4 раз включительно.

19. Дана строка 'a1a a12a a123a a1234a a12345a'. Найти и вывести строки,

где по краям стоит 'a', а между ними – **только** цифры, но не более 3-х. 20. Дана строка 'aAa aBa aCa aDa aEa aFa aGa'. Найти и вывести строки, где по краям стоит 'a', а между ними – **одна** заглавная латинская буква, **кроме** A, B, C.

21. Дана строка 'a1a a22a a333a a4444a a55555a'. Найти и вывести строки, где по краям стоит 'a', а между ними – **только** нечетные цифры (1, 3, 5, 7, 9) в любом количестве.

22. Дана строка 'a b a c a d a e a f'. Найти и вывести все подстроки, которые начинаются и заканчиваются на 'a', а между ними – ровно один пробел и одна любая буква.

23. Дана строка 'a1a a2b a3c a4d a5e'. Найти и вывести строки, где по краям стоят **разные** буквы (первая – 'a', последняя – любая от 'b' до 'e'), а между ними – одна цифра.

24. Дана строка 'a!a a@a a#a a$a a%a a^a'. Найти и вывести строки, где по краям стоит 'a', а между ними – **один** символ, который не является буквой и не является пробелом.

25. Дана строка 'aAa aBBa aCCCa aDDDDa'. Найти и вывести строки, где по краям стоит 'a', а между ними – **только** заглавные латинские буквы, но не менее 2-х и не более 4-х.

26. Дана строка 'a1a a12a a123a a1234a a12345a'. Найти и вывести строки, где по краям стоит 'a', а между ними – **только** цифры, и их количество четное (2, 4, 6...).

27. Дана строка 'a.a a..a a...a a....a a.....a'. Найти и вывести строки, где по краям стоит 'a', а между ними – от 2 до 5 точек включительно. 28. Дана строка 'a_a a__a a___a a____a a_____a'. Найти и вывести строки, где по краям стоит 'a', а между ними – символ подчеркивания (_) ровно 3 раза. 29. Дана строка 'a1a a22a a333a a4444a a55555a'. Найти и вывести строки, где по краям стоит 'a', а между ними – **только** цифры, но не содержащие цифру 0.

30. Дана строка 'aAa aBa aCa aDa aEa aFa aGa'. Найти и вывести строки, где по краям стоит 'a', а между ними – **одна** заглавная латинская буква от A до Z, но **не** гласная (A, E, I, O, U).

ДОМАШНЕЕ ЗАДАНИЕ

Индивидуальное задание

1. ЛИТЕРАТУРА

Блинов, И. Н., Романчик, В. С. Java from EPAM : учеб.-метод. пособие / И. Н. Блинов, В. С. Романчик. Минск: Четыре четверти, 2021. 560 с.

Преподаватель А.С.Кибисова